

Python important questions

Module 1

1. List and explain math operators used in python with example.

Table 1-1: Math Operators from Highest to Lowest Precedence

Operator	Operation	Example	Evaluates to...
**	Exponent	2 ** 3	8
%	Modulus/remainder	22 % 8	6
//	Integer division/floored quotient	22 // 8	2
/	Division	22 / 8	2.75
*	Multiplication	3 * 5	15
-	Subtraction	5 - 2	3
+	Addition	2 + 2	4

Precedence:

The ** operator is evaluated first;

The *, /, //, and % operators are evaluated next, from left to right;

The + and - operators are evaluated last (also from left to right).

Parentheses is used to override the usual precedence if needed.

Ex: (5 - 1) * ((7 + 1) / (3 - 1))

↓
4 * ((7 + 1) / (3 - 1))
↓
4 * (8) / (3 - 1)
↓
4 * (8) / (2)
↓
4 * 4.0
↓
16.0

2. What is a String? Explain String Concatination and Replication.

----- String is a collection of Characters.

String Concatenation.

When + is used on two string values, it joins the strings as the *string concatenation* operator.

Ex: 'Alice' + 'Bob'

'AliceBob'

Replication.

When the * operator is used on one string value and one integer value, it becomes the *string replication* operator.

'Alice' * 5

'AliceAliceAliceAliceAlice'

3. Write a python program and explain the following functions: int(), str(), float(), print(), len() and input().

print('Hello world!')
print('What is your name?')
myName = input()

```

print('It is good to meet you, ' + myName)
print("The length of your name is:")
print(len(myName))
print('What is your age?')
myAge = input()
print('You will be ' + str(int(myAge) + 1) + ' in a year.')

```

The print() Function

The print() function displays the string value inside the parentheses on the screen.

```

Ex: print('Hello world!')
      print('What is your name?')

```

The input() Function

The input() function waits for the user to type some text on the keyboard and press enter.

```

Ex: myName = input()
      The len() Function

```

You can pass the len() function a string value (or a variable containing a string), and the function evaluates to the integer value of the number of characters in that string.

```

Ex: print("The length of your name is:")
      print(len(myName))

```

The str()

The str() function can be passed an integer value and will evaluate to a string value version of it.

```

Ex: print('I am ' + str(29) + ' years old.')
      I am 29 years old.

```

int(), and float() Functions

int(), and float() functions will evaluate to the string, integer, and floating-point forms of the value you pass, respectively.

```

Ex: int(1.25)
      1
      float('3.14')
      3.14

```

4. Write a python program to check whether number is even or odd.

```

---- def evenOdd( x ):
    if (x %2==0):
        print("even")
    else:
        print("odd")
x=int(input("Enter the number"))
evenOdd(x)

```

5. With Python programming examples to each, explain the syntax and control flow diagrams of break and continue statements.

---- break

If the execution reaches a break statement, it immediately exits the while loop's clause. In code, a break statement simply contains the break keyword.

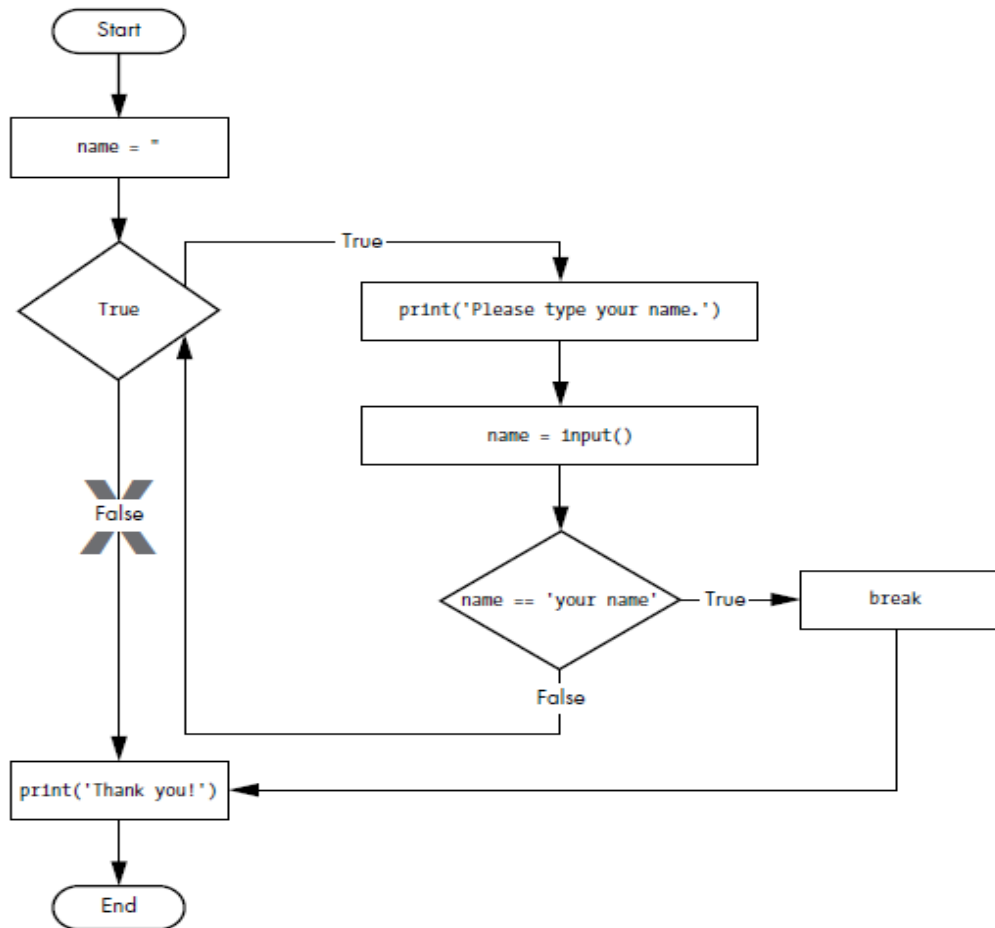
while True:

```

    print('Please type your name.')
    name = input()
    if name == 'your name':
        break

```

```
print('Thank you!')
```



Continue

Continue statements are used inside loops. When the program execution reaches a continue statement, execution immediately jumps back to the start of the loop and continues the loop's condition.

Eg: while True:

```
    print('Who are you?')
```

```
    name = input()
```

```
    if name != 'Joe':
```

```
        continue
```

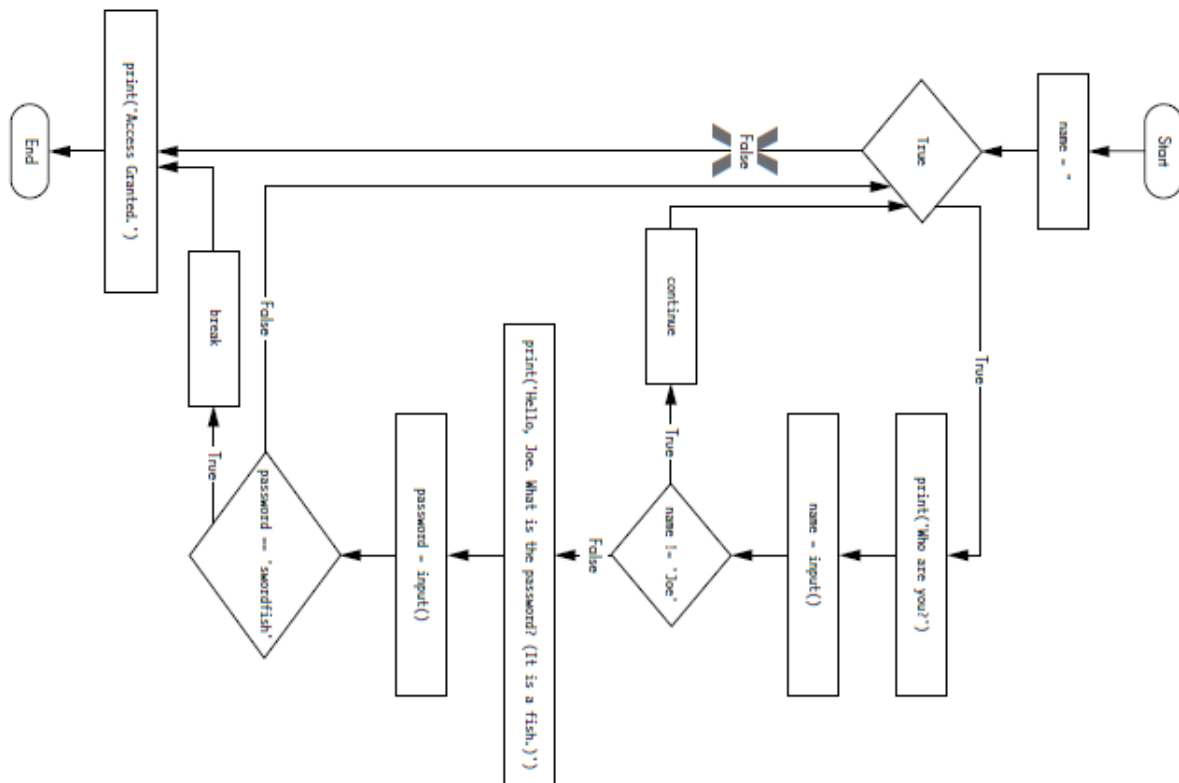
```
    print('Hello, Joe. What is the password? (It is a fish.)')
```

```
    password = input()
```

```
    password == 'swordfish':
```

```
        break
```

```
print('Access granted.')
```



6. Explain TWO ways of importing modules into application in Python with syntax and suitable programming examples.

-----**(i).**Before using functions in a module, we must import the module with an import statement. In code:

The **import** keyword

The name of the module

Optionally, more module names, as long as they are separated by commas.

Eg: import random

for i in range(5):

 print(random.randint(1, 10))

(ii).An alternative form of the import statement is composed of the **from** keyword, followed by the module name, the **import** keyword, and a star; for example, **from random import ***.

7. Write a function to calculate factorial of a number. Develop a program to compute binomial coefficient (Given N and R).

----- def factorial(z):

 if z==1:

 return 1

 else:

 return z* factorial(z-1)

def binomial_coefficient(n,k):

 a= (factorial(n) / factorial(k) * factorial(n-k))

 return a

n=int(input("enter a number to find factorial"))

print("The factorial of number "+str(n) +"is",factorial(n))

k=int(input("enter the number to find binomial coefficient"))

print("The binomial coefficient is:",binomial_coefficient(n,k))

Output

enter a number to find factorial4

The factorial of number 4is 24

enter the number to find binomial coefficient2

The binomial coefficient is: 6.0

8. Explain looping control statements in Python with a syntax and example to each.

----(i)while Loop:

The code in a while clause will be executed as long as the while statement's condition is True.

The condition is always checked at the start of each iteration. If the condition is False, the while clause is skipped.

A **while** statement always consists of the following:

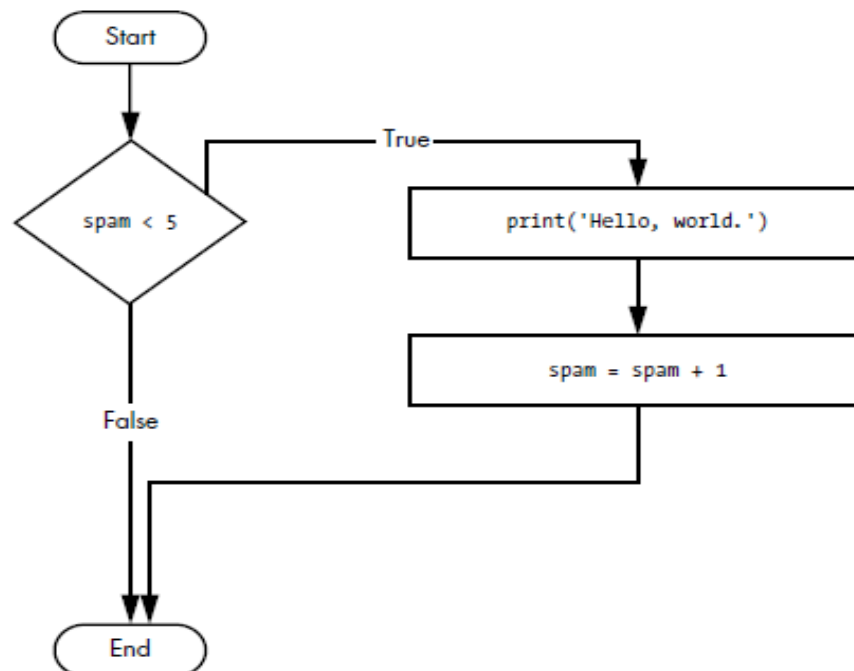
The while keyword

A condition (that is, an expression that evaluates to True or False)

A colon

Starting on the next line, an indented block of code (called the whileclause)

Example: spam = 0
while spam < 5:
 print('Hello, world.')
 spam = spam + 1



(ii) for Loops and the range() Function:

for loop statement and the range() function is used to execute a block of code only a certain number of times In code, a for includes the following:

The for keyword

A variable name

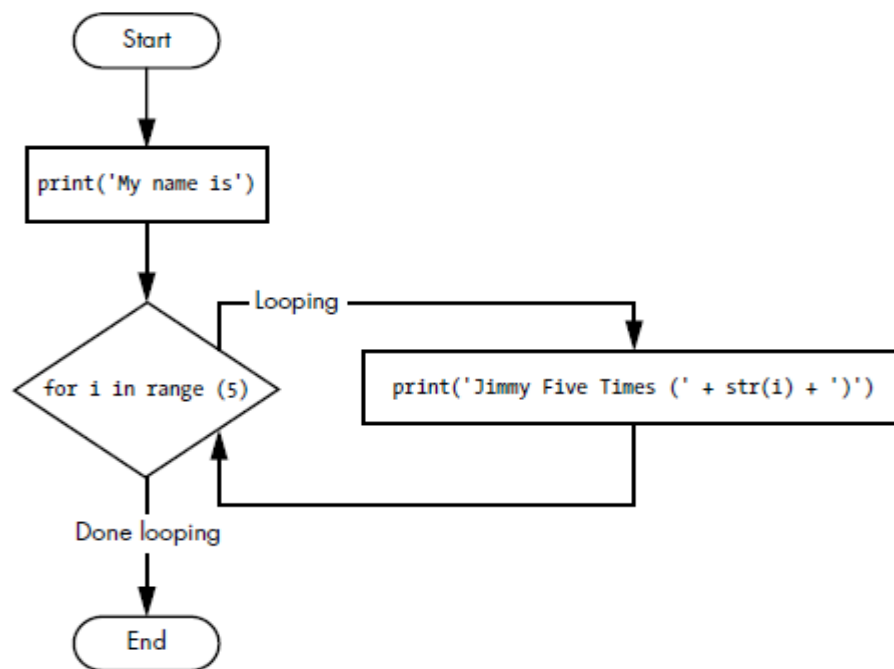
The in keyword

A call to the range() method with up to three integers passed to it

A colon

Starting on the next line, an indented block of code (called the forclause)

Example: `print('My name is')`
`for i in range(5):`
`print('Jimmy Five Times (' + str(i) + '')`



9. With an example explain user defined functions.

----- All the functions that are written by any of us come under the category of user-defined functions. Below are the steps for writing user-defined functions in Python.

- In Python, a **def keyword** is used to declare user-defined functions.
- An indented block of statements follows the function name and arguments which contains the body of the function.

Syntax:

```
def function_name():
    statements
```

The function may take **arguments(s)** also called parameters as input within the opening and closing parentheses, just after the function name followed by a colon.

Syntax:

```
def function_name(argument1, argument2, ...):
    statements
```

An **argument** is a parameter that assumes a default value if a value is not provided in the function call for that argument.

```
Ex: def evenOdd(x):
if (x %2==0):
print("even")
else:
print("odd")
x=int(input("Enter the number"))
evenOdd(x)
```

10. Develop a Python program to generate Fibonacci sequence of length (N). Read N from the console.

----- `def fib(nterms):`

```

n1, n2 = 0, 1
count = 0
if nterms <= 0:
    print("Please enter a positive integer")
else:
    print("Fibonacci sequence for number "+str(nterms)+" is")
    while count < nterms:
        print(n1)
        nth = n1 + n2
        n1 = n2
        n2 = nth
        count += 1
number = int(input("How many terms? "))
fib(number)

```

Output:

```

How many terms? 5
Fibonacci sequence for number 5 is
0
1
1
2
3

```

11. Explain FOUR scope rules of variables in Python.

-----1. Local Variables Cannot Be Used in the Global Scope.

When the program execution is in the global scope, no local scopes exist. This is why only global variables can be used in the global scope

Ex: def spam():
 eggs = 31337

```

spam()
print(eggs)

```

The error happens because the eggs variable exists only in the local scope created when spam() is called. Once the program execution returns from spam, that local scope is destroyed, and there is no longer a variable named eggs.

2. Local Scopes Cannot Use Variables in Other Local Scopes.

The local variables in one function are completely separate from the local variables in another function.

Ex:def spam():
 eggs = 99
 bacon()
 print(eggs)
def bacon():
 ham = 101
 eggs = 0
spam()

3. Global Variables Can Be Read from a Local Scope

```

def spam():
    print(eggs)
eggs = 42
spam()
print(eggs)

```

Since there is no parameter named eggs or any code that assigns eggs a value in the spam() function, when eggs is used in spam(), Python considers it a reference to the global variable eggs. This is why 42 is printed when the previous program is run.

4. Local and Global Variables with the Same Name

Avoid using local variables that have the same name as a global variable or another local variable.

```
def spam():
    eggs = 'spam local'
    print(eggs) # prints 'spam local'
def bacon():
    eggs = 'bacon local'
    print(eggs) # prints 'bacon local'
    spam()
    print(eggs) # prints 'bacon local'
eggs = 'global'
bacon()
print(eggs) # prints 'global'
```

O/P:

bacon local
spam local
bacon local
global

12. Explain local and global scope in python.

----- **Local Scope**

- A local scope is created whenever a function is called.
- Parameters and variables that are assigned in function call termed as *function's local scope*.
- A variable that exists in a local scope is called a *local variable*.
- Any variables assigned in this function exist within the local scope.
- When the function returns, the local scope is destroyed, and these variables are forgotten.
- A local scope can access global variables.

```
Ex: def spam():
    eggs = 99
    bacon()
    print(eggs)
```

Global Scope

- Variables that are assigned outside all functions are said to exist in the *global scope*.
- A variable that exists in the global scope is called a *global variable*.
- There is only one global scope, and it is created when your program begins. When your program terminates, the global scope is destroyed, and all its variables are forgotten.
- Code in the global scope cannot use any local variables.

```
Ex: def spam():
    print(eggs)
eggs = 42
spam()
print(eggs)
```

13. Define exception handling. How exception handling is done in python with program to solve divide by zero exception.

----- Exception Handling is a process of detecting errors and handling them. If any error occurs in program we don't want our program to crash instead the program must detect errors, handle them, and continue to run.

Errors can be handled with try and except statements. The code is put in a try clause. The program execution moves to the start of an except clause if an error happens.

Program: divide by zero exception.

```
def spam(divideBy):
```



```

    try:
        return 42 / divideBy
    except ZeroDivisionError:
        print('Error: Invalid argument.')
print(spam(2))
print(spam(12))
print(spam(0))
print(spam(1))
O/P
21.0
3.5
Error: Invalid argument.
None
42.0

```

14. What are functions? Explain python functions with parameters and return value.

----- **A function** is like a mini-program within a program. A major purpose of functions is to group code that gets executed multiple times.

A parameter is a variable that an argument is stored in when a function is called. One special thing to note about parameters is that the value stored in a parameter is forgotten when the function returns.

```

Ex:  def hello(name):
        print('Hello ' + name)
hello('Alice')
hello('Bob')

```

Return Values and return Statements.

The value that a function call evaluates to is called the **return value** of the function.

We can specify return value with a **return statement**. A return statement consists of the following:

The return keyword

The value or expression that the function should return.

```

Ex:  import random
def getAnswer(answerNumber):
    if answerNumber == 1:
        return 'It is certain'

```

Note: Read the sample programs which were initially executed in the lab .

Module 2

1. Explain negative indexing, slicing(), append(), index(), insert(), sort(), remove(), reverse() methods with respect to lists in Python.

----- (i). **negative indexing:** The integer value -1 refers to the last index in a list, the value -2 refers to the second-to-last index in a list, and so on.

```

Ex: spam = ['cat', 'bat', 'rat', 'elephant']
>>> spam[-1]
'elephant'

```

(ii). **slicing():** A slice is typed between square brackets, like an index, but it has two integers separated by a colon.

```

Ex: spam = ['cat', 'bat', 'rat', 'elephant']
>>> spam[0:4]
['cat', 'bat', 'rat', 'elephant']

```

(iii). append():To add new values to a list, use the `append()` method that adds the argument to the end of the list.

```
Ex: spam = ['cat', 'dog', 'bat']
>>>spam.append('moose')
Print(spam)
['cat', 'dog', 'bat', 'moose']
```

(iv). index():List have an `index()` method where if the value exists in the list, the index of the value is returned. If not, then Python produces a `ValueError` error.

```
Ex: spam = ['hello', 'hi', 'howdy', 'heyas']
>>>spam.index('hello')
0
```

(v). insert():The `insert()` method can insert a value at any index in the list.

The first argument to `insert()` is the index for the new value, and the second argument is the new value to be inserted.

```
Ex: spam = ['cat', 'dog', 'bat']
>>>spam.insert(1, 'chicken')
>>>spam
['cat', 'chicken', 'dog', 'bat']
```

(vi). Sort():Lists of number values or lists of strings can be sorted with the `sort()` method.

```
Ex: spam = [2, 5, 3.14, 1, -7]
>>>spam.sort()
>>>spam
[-7, 1, 2, 3.14, 5]
```

(vii). remove():The `remove()` method is passed the value to be removed from the list it is called on.

```
Ex: spam = ['cat', 'bat', 'rat', 'elephant']
>>>spam.remove('bat')
>>>spam
['cat', 'rat', 'elephant']
```

(viii). reverse():The `reverse` keyword argument to have `sort()` function sort the values in reverse order.

```
Ex: spam.sort(reverse=True)
>>>spam
['elephants', 'dogs', 'cats', 'badgers', 'ants']
```

2. Develop suitable Python programs with nested lists to explain `copy.copy()` and `copy.deepcopy()` functions.

----- Python provides a module named `copy` that provides both the `copy()` and `deepcopy()` functions.

(i). Copy(): `copy.copy()`, can be used to make a duplicate copy of a mutable value like a list or dictionary, not just a copy of a reference.

```
Ex: import copy
>>>spam = ['A', 'B', 'C', 'D']
>>>cheese = copy.copy(spam)
>>>cheese[1] = 42
>>>spam
['A', 'B', 'C', 'D']
>>>cheese
['A', 42, 'C', 'D']
```

(ii). Deepcopy(): If the list you need to copy contains lists, then use the `copy.deepcopy()`

function instead of `copy.copy()`. The `deepcopy()` function will copy these inner lists as well.

```
Ex: import copy
>>>spam = ['A', 'B', 'C', 'D']
>>>cheese = copy.copy(spam)
>>>cheese[1] = 42
>>>spam
['A', 'B', 'C', 'D']
>>>cheese
['A', 42, 'C', 'D']
```

3. Explain different ways to delete an element from a list with suitable Python syntax and programming examples.

----- We have two methods to delete elements in list.

(i). `del()`: Del method is used to delete an element from the list by giving index.

```
Ex: eggs = [1, 2, 3]
>>>del eggs[2]
>>>del eggs[1]
>>>del eggs[0]
eggs
[]
```

(ii). `remove()`: The `remove()` method is passed the value to be removed from the list it is called on.

```
Ex: spam = ['cat', 'bat', 'rat', 'elephant']
>>>spam.remove('bat')
>>>spam
['cat', 'rat', 'elephant']
```

4. Tuples are immutable. Explain with Python programming example.

-----The *tuple* data type is typed with parentheses, (and), instead of square brackets.

Tuples are immutable. Tuples cannot have their values modified, appended, or removed.

```
Ex: eggs = ('hello', 42, 0.5)
>>>eggs[1] = 99
Traceback (most recent call last):
File "<pyshell#5>", line 1, in <module>
eggs[1] = 99
TypeError: 'tuple' object does not support item assignment
```

5. Explain how lists are mutable.

-----A list value is a *mutable* data type: It can have values added, removed, or changed.

```
Ex: spam = ['cat', 'bat', 'rat', 'elephant']
>>>spam.remove('bat')
>>>spam
['cat', 'rat', 'elephant']
Ex: spam = ['cat', 'dog', 'bat']
>>>spam.insert(1, 'chicken')
>>>spam
```

```
['cat', 'chicken', 'dog', 'bat']
```

6. Explain with a programming example to each: (ii) `get()` (iii) `setdefault()`

----- `get()`: dictionaries have a `get()` method that takes two arguments: the key of the value to retrieve and a fallback value to return if that key does not exist.

```
Ex: picnicItems = {'apples': 5, 'cups': 2}
>>>'I am bringing ' + str(picnicItems.get('cups', 0)) + ' cups.'
'I am bringing 2 cups.'
>>>'I am bringing ' + str(picnicItems.get('eggs', 0)) + ' eggs.'
```

'I am bringing 0 eggs.'

Setdefault(): In setdefault method The first argument passed to the method is the key to check for, and the second argument is the value to set at that key if the key does not exist. If the key does exist, the setdefault() method returns the key's value.

```
Ex: d={}
for i in num:
    d.setdefault(i,0)
    d[i]=d[i]+1
print(d)
for i in d:
    print('Frequency of '+str(i)+' is '+str(d[i]))
```

Output:

```
Enter a multidigit number: 12342312445788
{'1': 2, '2': 3, '3': 2, '4': 3, '5': 1, '7': 1, '8': 2}
Frequency of 1 is 2
Frequency of 2 is 3
Frequency of 3 is 2
Frequency of 4 is 3
Frequency of 5 is 1
Frequency of 7 is 1
Frequency of 8 is 2
```

7. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with suitable message.

```
-----d={}
for i in num:
    d.setdefault(i,0)
    d[i]=d[i]+1 print(d)
for i in d:
    print('Frequency of '+str(i)+' is '+str(d[i]))
```

Output:

```
Enter a multidigit number: 12342312445788
{'1': 2, '2': 3, '3': 2, '4': 3, '5': 1, '7': 1, '8': 2}
Frequency of 1 is 2
Frequency of 2 is 3
Frequency of 3 is 2
Frequency of 4 is 3
Frequency of 5 is 1
Frequency of 7 is 1
Frequency of 8 is 2
```

8. Program to find mean, variance and standard deviation.

```
-----import math
data=[1,4,3,5,6]
def mean(data):
    n=len(data)
    mean=sum(data)/n
    return mean
print("the mean is" + str (mean(data)))
def variance(data):
    n=len(data)
    mean=sum(data)/n
```

```

        dev=[(x-mean)**2 for x in data]
        var=sum(dev)/n
        return var
    print("the variance is "+str(variance(data)))
    def standardDev(data):
        var=variance(data)
        std=math.sqrt(var)
        return std
    print("the standard deviation is "+str(standardDev(data)))

```

Output:

the mean is 3.8

the variance is 2.96

the standard deviation is 1.7204650534085253

9. Explain use of in and not in operators.

----- We can determine whether a value is or isn't in a list with the in and not in operators. in and not in are used in expressions and connect two values.

```

Ex: 'howdy' in ['hello', 'hi', 'howdy', 'heyas']
True
>>> spam = ['hello', 'hi', 'howdy', 'heyas']
>>> 'cat' in spam
False
>>> 'howdy' not in spam
False
>>> 'cat' not in spam
True

```

10. Write a program to count the frequency of characters using module PrettyPrint(PPrint).

----- The pprint() and pformat() functions that will "pretty print" a dictionary's values. This is helpful when you want a cleaner display of the items in a dictionary than what print() provides.

Ex: import pprint

```

message = 'It was a bright cold day in April, and the clocks were striking
thirteen.'
count = {}
for character in message:
    count.setdefault(character, 0)
    count[character] = count[character] + 1
pprint.pprint(count)

```

O/P:

```

{' ': 13,
 ',': 1,
 '.': 1,
 'A': 1,
 'I': 1,
 'a': 4,
 'b': 1,
 'c': 3,
 'd': 3,
 'e': 5,
 'g': 2,
 'h': 3,

```

```
'i': 6,  
'k': 2,  
'l': 3,  
'n': 4,  
'o': 2,  
'p': 1,  
'r': 5,  
's': 3,  
't': 6,  
'w': 2,  
'y': 1}
```

Module 3

1. Explain Python string handling methods with examples: `split()`, `endswith()`, `ljust()`, `center()`, `lstrip()`, `join()`, `startswith()`, `rjust()`, `strip()`, `rstrip()`.

-----i. **join()**: The `join()` method is useful when you have a list of strings that need to be joined together into a single string value.

```
Ex: ', '.join(['cats', 'rats', 'bats'])  
'cats, rats, bats'
```

- ii. **split()**: The `split()` method is called on a string value and returns a list of strings.

```
Ex: 'My name is Simon'.split()  
['My', 'name', 'is', 'Simon']
```

- iii. **endswith()**, **startswith()**: The `startswith()` and `endswith()` methods return `True` if the string value they are called on begins or ends (respectively) with the string passed to the method; otherwise, they return `False`.

```
Ex: 'Hello world!'.startswith('Hello')  
True  
>>> 'Hello world!'.endswith('world!')  
True
```

- iv. **strip()**, **rstrip()**, **lstrip()**: The `strip()` string method will return a new string without any whitespace characters at the beginning or end. The **`lstrip()` and `rstrip()` methods** will remove whitespace characters from the left and right ends, respectively

```
Ex: spam = ' Hello World '  
>>> spam.strip()  
'Hello World'  
>>> spam.lstrip()  
'Hello World '  
>>> spam.rstrip()  
' Hello World'
```

- v. **ljust()**, **rjust()**, **center()**: The `rjust()` and `ljust()` string methods return a padded version of the string they are called on, with spaces inserted to justify the text.

```
Ex: 'Hello World'.rjust(20)  
' Hello World'  
>>> 'Hello'.ljust(10)  
'Hello '
```

The `center()` string method works like `ljust()` and `rjust()` but centers the text rather than justifying it to the left or right

```
Ex: 'Hello'.center(20)  
' Hello '
```

2. Explain the concept of file path. Also explain absolute and relative path.

-----A file has two key properties: a filename (usually written as one word) and a path. The path specifies the location of a file on the computer.

It has directories, folders and files.

Ex: c:\users\documents\file.txt

Absolute path: An absolute path, which always begins with the root folder. There are also the dot (.) folders.

Calling `os.path.abspath(path)` will return a string of the absolute path of the argument. This is an easy way to convert a relative path into an absolute one.

Ex: `os.path.abspath('.')`
'C:\\Python34'

Relative path(): A relative path, which is relative to the program's current working directory. There are also the dot-dot (..) folders.

Calling `os.path.relpath(path, start)` will return a string of a relative path from the start path to path. If start is not provided, the current working directory is used as the start path.

Ex: `os.path.relpath('C:\\Windows', 'C:\\')`
'Windows'

3. Explain with suitable Python program segments: (i) `os.path.basename()` (ii) `os.path.join()`. iii. `os.path.dirname()`

-----**(i) `os.path.basename()`:** Calling `os.path.basename(path)` will return a string of everything that comes after the last slash in the path argument.

Ex: `path = 'C:\\Windows\\System32\\calc.exe'`
`>>>os.path.basename(path)`
'calc.exe'

(ii) `os.path.join()`: `os.path.join()` will return a string with a file path using the correct path separators.

Ex: `import os`
`>>>os.path.join('usr', 'bin', 'spam')`
'usr\\bin\\spam'

iii. `os.path.dirname()`: Calling `os.path.dirname(path)` will return a string of everything that comes before the last slash in the path argument.

Ex: `>>>os.path.dirname(path)`
'C:\\Windows\\System32'

4. Explain reading and saving python program variables using shelve module with suitable Python program.

-----You can save variables in your Python programs to binary shelf files using the shelve module.

The shelve module will let you add Save and Open features to your program.

To read and write data using the shelve module, you first import shelve. Call `shelve.open()` and pass it a filename, and then store the returned shelf value in a variable.

Ex: `import shelve`
`>>>shelfFile = shelve.open('mydata')`
`>>>cats = ['Zophie', 'Pooka', 'Simon']`
`>>>shelfFile['cats'] = cats`
`>>>shelfFile.close()`

5. Develop a Python program to read and print the contents of a text file.

-----`list1=[]`
`file=open("D:\\hello1.txt")`
`var=file.readlines()`
`print(var)`
`for i in var:`
`list1.append(i.strip())`
`print(list1)`
`list1.sort()`
`print(list1)`
`file.close()`
`file1=open("D:\\hello2.txt",'w')`
`for i in list1:`

```

        file1.write(i+'\n')
file1.close()

```

Output:

```

['vidya\n', 'anugna\n', 'shruthi\n', 'bindu\n']
Hello1.txt:['vidya', 'anugna', 'shruthi', 'bindu']
Hello2.txt:['anugna', 'bindu', 'shruthi', 'vidya']

```

6. Develop a Python program find the total size of all the files in the given.

-----Calling os.path.getsize(path) will return the size in bytes of the file in the path argument.

Program:

```

totalSize = 0
    for filename in os.listdir('C:\\Windows\\System32'):
totalSize = totalSize + os.path.getsize(os.path.join('C:\\Windows\\System32', filename))
print(totalSize)

```

7. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file.

```

----- list1=[]
        file=open("D:\\hello1.txt")
        var=file.readlines()
        print(var)
        for i in var:
            list1.append(i.strip())
        print(list1)
        list1.sort()
        print(list1)
        file.close()
        file1=open("D:\\hello2.txt",'w')
        for i in list1:
            file1.write(i+'\n')
        file1.close()

```

Output:

```

['vidya\n', 'anugna\n', 'shruthi\n', 'bindu\n']
Hello1.txt:['vidya', 'anugna', 'shruthi', 'bindu']
Hello2.txt:['anugna', 'bindu', 'shruthi', 'vidya']

```

8. Explain with example isalpha(), isalnum(), isspace(), isdecimal(), isupper(), islower().

-----**(i). isupper(), islower():**The isupper() and islower() methods will return a Boolean True value if the string has at least one letter and all the letters are uppercase or lowercase, respectively.

Ex: spam = 'Hello world!'

```

>>>spam.islower()
False
>>>'HELLO'.isupper()
True

```

ii. isalpha(): isalpha() returns True if the string consists only of letters and is not blank.

```

>'hello'.isalpha()
True
>>>'hello123'.isalpha()
False

```

iii. isalnum(): isalnum() returns True if the string consists only of letters and numbers and is not blank.

```

'hello123'.isalnum()
True

```

iv. isdecimal():isdecimal() returns True if the string consists only of numeric characters and is not blank.

```

'123'.isdecimal()

```


True

v. isspace(): returns True if the string consists only of spaces, tabs, and newlines and is not blank.

```
>>> ''.isspace()
```

True

vi. istitle(): istitle() returns True if the string consists only of words that begin with an uppercase letter followed by only lowercase letters.

```
>>> 'This Is Title Case'.istitle()
```

True

9. Write a program to accept string and display total number of alphabets.

```
-----def count_alphabets(input_string):
    alphabet_count = 0
    for char in input_string:
        if char.isalpha(): # check if the character is alphabetic
            alphabet_count += 1
    return alphabet_count

input_string = input("Enter a string: ")
num_alphabets = count_alphabets(input_string)
print(f"Total number of alphabets in the string: {num_alphabets}")
```

10. Develop a python code to determine whether give string is a palindrome or not.

```
-----def isPalindrome(s):
    return s == s[::-1]
s = "malayalam"
ans = isPalindrome(s)
if ans:
    print("Yes")
else:
    print("No")
```

Module 4

1. Explain permanent delete and safe delete with a suitable Python programming example to each.

----- **Permanently Deleting Files:**

To delete a folder and all of its contents, you use the `shutil` module.

- Calling `os.unlink(path)` will delete the file at *path*.
- Calling `os.rmdir(path)` will delete the folder at *path*. This folder must be empty of any files or folders.
- Calling `shutil.rmtree(path)` will remove the folder at *path*, and all files and folders it contains will also be deleted.

```
Ex: import os
for filename in os.listdir():
    if filename.endswith('.txt'):
        os.unlink(filename)
```

Safe Deletes with the send2trash Module:

A much better way to delete files and folders is `send2trash` module.

`send2trash` will send folders and files to your computer's trash or recycle bin instead of permanently deleting them.

```
Ex: import send2trash
>>> baconFile = open('bacon.txt', 'a') # creates the file
>>> baconFile.write('Bacon is not a vegetable.')
25
>>> baconFile.close()
>>> send2trash.send2trash('bacon.txt')
```

2. Develop a program to backing Up a given Folder (Folder in a current working directory) into a ZIP File by using relevant modules and suitable methods.

```
----- import zipfile, os
def backupToZip(folder):
    folder = os.path.abspath(folder) # make sure folder is absolute
    number = 1
    while True:
        zipFilename = os.path.basename(folder) + '_' + str(number) + '.zip'
        if not os.path.exists(zipFilename):
            break
        number = number + 1
    print('Creating %s...' % (zipFilename))
    backupZip = zipfile.ZipFile(zipFilename, 'w')
    for foldername, subfolders, filenames in os.walk(folder):
        print('Adding files in %s...' % (foldername))
        backupZip.write(foldername)
        for filename in filenames:
            if filename.startswith(os.path.basename(folder) + '_') and
                filename.endswith('.zip'):
                continue
        backupZip.write(os.path.join(foldername, filename))
    backupZip.close()
    print('Done.')
```

```
backupToZip('C:\\Users\\PC\\AppData\\Local\\Programs\\Python\\Python311\\PES')
```

Output:

```
Creating PES_1.zip...
Adding files in C:\\Users\\PC\\AppData\\Local\\Programs\\Python\\Python311\\PES...
Adding files in C:\\Users\\PC\\AppData\\Local\\Programs\\Python\\Python311\\PES\\pes1...
Done.
```

3. Explain the role of Assertions in Python with a suitable program.

----- An *assertion* is a sanity check to make sure your code isn't doing something obviously wrong. These sanity checks are performed by `assert` statements. If the sanity check fails, then an `AssertionError` exception is raised.

`assert` statement consists of the following:

- The `assert` keyword
- A condition (that is, an expression that evaluates to `True` or `False`)
- A comma
- A string to display when the condition is `False`

Ex: `podBayDoorStatus = 'open'`

```
>>> assert podBayDoorStatus == 'open', 'The pod bay doors need to be "open".'
```

```
>>> podBayDoorStatus = 'I'm sorry, Dave. I'm afraid I can't do that.'
```

```
>>> assert podBayDoorStatus == 'open', 'The pod bay doors need to be "open".'
```

Traceback (most recent call last):

`AssertionError: The pod bay doors need to be "open".`

4. Develop a program with function named `DivExp` which takes two parameters `a`, `b` and `c`. Write assertion for `a>0` and raise exception for `b=0`.

```
----- def DivExp(a,b):
    assert a>0,"a is greater than zero"
    if b==0:
        raise Exception("value of b cannot be zero")
    else:
        c=a/b
    return c
a=int(input("enter the value of A"))
b=int(input("enter the value of B"))
try:
    print(DivExp(a,b))
except Exception as err:
    print(err)
```

Output:

```
enter the value of A4
enter the value of B0
value of b cannot be zero
```

5. Explain the functions with examples: (i) `shutil.copytree()`, (ii) `shutil.copy`, (iii) `shutil.move()` (iv) `shutil.rmtree()`.

----- The `shutil` module provides functions for copying files, as well as entire folders.

(i). `shutil.copy()`: Calling `shutil.copy(source, destination)` will copy the file at the path source to the folder at the path destination.

Ex: `shutil.copy('C:\\spam.txt', 'C:\\delicious')`
`'C:\\delicious\\spam.txt'`

(ii) `shutil.copytree()`: The `shutil.copytree()` call creates a new folder named `bacon_backup` with

the same content as the original *bacon* folder.

```
Ex: import shutil, os
>>> os.chdir('C:\\')
>>> shutil.copytree('C:\\bacon', 'C:\\bacon_backup')
'C:\\bacon_backup'
```

(iii). **shutil.move()**: Calling `shutil.move(source, destination)` will move the file or folder at the path

source to the path *destination* and will return a string of the absolute path of the new location.

```
Ex: import shutil
>>> shutil.move('C:\\bacon.txt', 'C:\\eggs')
'C:\\eggs\\bacon.txt'
```

6. Develop a Python program to traverse the current directory by listing subfolders and files.

----- The `os.walk()` function is passed a single string value: the path of a folder.

```
import os
for folderName, subfolders, filenames in os.walk('C:\\delicious'):
    print('The current folder is ' + folderName)
for subfolder in subfolders:
    print('SUBFOLDER OF ' + folderName + ':' + subfolder)
for filename in filenames:
    print('FILE INSIDE ' + folderName + ':' + filename)
print("")
```

7. Explain the support for Logging with logging module in Python.

----- Python's logging module makes it easy to create a record of custom messages that you write.

These log messages will describe when the program execution has reached the logging function call and list any variables you have specified at that point in time.

```
Ex: import logging
logging.basicConfig(level=logging.DEBUG, format='%(asctime)s - %(levelname)s - %(message)s')
logging.debug('Start of program')
```

```
def factorial(n):
    logging.debug('Start of factorial(%s%%)' % (n))
    total = 1
    for i in range(n + 1):
        total *= i
        logging.debug('i is ' + str(i) + ', total is ' + str(total))
    logging.debug('End of factorial(%s%%)' % (n))
    return total
print(factorial(5))
logging.debug('End of program')
```

```
O/P: 2015-05-23 16:20:12,664 - DEBUG - Start of program
2015-05-23 16:20:12,664 - DEBUG - Start of factorial(5)
2015-05-23 16:20:12,665 - DEBUG - i is 0, total is 0
2015-05-23 16:20:12,668 - DEBUG - i is 1, total is 0
2015-05-23 16:20:12,670 - DEBUG - i is 2, total is 0
2015-05-23 16:20:12,673 - DEBUG - i is 3, total is 0
2015-05-23 16:20:12,675 - DEBUG - i is 4, total is 0
2015-05-23 16:20:12,678 - DEBUG - i is 5, total is 0
2015-05-23 16:20:12,680 - DEBUG - End of factorial(5)
```

0

8. Explain logging levels in logging module.

----- *Logging levels* provide a way to categorize your log messages by importance. There are five logging levels:

Level	Logging Function	Description
DEBUG	<code>logging.debug()</code>	The lowest level. Used for small details. Usually you care about these messages only when diagnosing problems.
INFO	<code>logging.info()</code>	Used to record information on general events in your program or confirm that things are working at their point in the program.
WARNING	<code>logging.warning()</code>	Used to indicate a potential problem that doesn't prevent the program from working but might do so in the future.
ERROR	<code>logging.error()</code>	Used to record an error that caused the program to fail to do something.
CRITICAL	<code>logging.critical()</code>	The highest level. Used to indicate a fatal error that has caused or is about to cause the program to stop running entirely.

9. Explain Idle debug control window.

----- The *debugger* is a feature of IDLE that allows you to execute your program one line at a time. The debugger will run a single line of code and then wait for you to tell it to continue. The program will stay paused until you press one of the five buttons in the Debug Control window: **Go, Step, Over, Out, or Quit.**

(i). **Go:** Clicking the Go button will cause the program to execute normally until it terminates or reaches a *breakpoint*.

(ii). **Step:** Clicking the Step button will cause the debugger to execute the next line of code and then pause again. The Debug Control window's list of global and local variables will be updated if their values change. If the next line of code is a function call, the debugger will "step into" that function and jump to the first line of code of that function.

(iii). **Over** Clicking the Over button will execute the next line of code. The function's code will be executed at full speed, and the debugger will pause as soon as the function call returns.

(iv). **Out:** Clicking the Out button will cause the debugger to execute lines of code at full speed until it returns from the current function.

(v). **Quit:** If you want to stop debugging entirely and not bother to continue executing the rest of the program, click the **Quit** button. The Quit button will immediately terminate the program.

Module 5

1. Explain the methods `__init__` and `__str__` with suitable code example to each.

----- `__init__` :

The `init` method (short for “initialization”) is a special method that gets invoked when an object is instantiated. Its full name is `__init__` (two underscore characters, followed by `init`, and then two more underscores). An `init` method for the `Time` class might look like this:

```
class Time:
    def __init__(self, hour=0, minute=0, second=0):
        self.hour = hour
        self.minute = minute
        self.second = second
```

The `__str__` method:

`__str__` is a special method, like `__init__`, that is supposed to return a string representation of an object.

Ex: inside class `Time`:

```
def __str__(self):
    return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)
```

When you print an object, Python invokes the `str` method:

```
>>> time = Time(9, 45)
>>> print(time)
09:45:00
```

2. Explain the program development concept ‘prototype and patch’ with suitable example.

----- For each function, a prototype is written that performs the basic calculation and then tested it, patching errors along the way. This is called prototype and patch.

Ex: Here is a function that converts `Times` to integers:

```
def time_to_int(time):
    minutes = time.hour * 60 + time.minute
    seconds = minutes * 60 + time.second
    return seconds
```

And here is a function that converts an integer to a `Time`

```
def int_to_time(seconds):
    time = Time()
    minutes, time.second = divmod(seconds, 60)
    time.hour, time.minute = divmod(minutes, 60)
    return time
```

3. Define a function which takes TWO objects representing complex numbers and returns new complex number with a addition of two complex numbers. Define a suitable class ‘Complex’ to represent the complex number. Develop a program to read N (N >=2) complex numbers and to compute the addition of N complex numbers

----- class `Complex`:

```
def sum(a,b):
    c=Complex()
    c.real=a.real+b.real
```

```

        c.img=a.img+b.img
    return c
a=Complex()
a.real=int(input('Enter the real part of a: '))
a.img=int(input('Enter the img part of a: '))
b=Complex()
b.real=int(input('Enter the real part of b: '))
b.img=int(input('Enter the img part of b: '))
c=sum(a,b)
print(c.real,'+',c.img,'j')

```

Output:

```

Enter the real part of a:2
Enter the img part of a:3
Enter the real part of b:4
Enter the img part of b:5
6+9j

```

4. Explain the following with syntax and suitable code snippet: i) Class definition ii) instantiation iii) passing an instance (or objects) as an argument iv) instances as return values.

----- **i) Class definition:** A class is a programmer-defined which acts as a blue print or a prototype. A class definition looks like this:

```

class Point:
    """Represents a point in 2-D space."""
    The header indicates that the new class is called Point. The body is a docstring that explains
    what the class is for.

```

ii) instantiation: Creating a new object is called instantiation, and the object is an instance of the class. When instance is printed, Python tells you what class it belongs to and where it is stored in memory.

```

Ex: blank = Point()
>>> blank

```

iii) instances as return values: Functions can return instances. For example, find_center takes a Rectangle as an argument and returns a Point that contains the coordinates of the center of the Rectangle:

```

def find_center(rect):
    p = Point()
    p.x = rect.corner.x + rect.width/2
    p.y = rect.corner.y + rect.height/2
    return p

```

iv). passing an instance (or objects) as an argument:

You can pass an instance as an argument in the usual way. For example:

```

def print_point(p):
    print('(%g, %g)' % (p.x, p.y))

```

print_point takes a point as an argument and displays it in mathematical notation. To invoke it, you can pass blank as an argument:

```

>>> print_point(blank)
(3.0, 4.0)

```

5. Define pure function and modifier. Explain the role of pure functions and modifiers in application development with suitable python programs.

----- **Pure functions:**

pure function because it does not modify any of the objects passed to it as arguments and it

has no effect, like displaying a value or getting user input, other than returning a value.

```
Ex: def add_time(t1, t2):
    sum = Time()
    sum.hour = t1.hour + t2.hour
    sum.minute = t1.minute + t2.minute
    sum.second = t1.second + t2.second
    return sum
```

Modifiers:

Sometimes it is useful for a function to modify the objects it gets as parameters. In that case, the changes are visible to the caller. Functions that work this way are called **modifiers**.

```
Ex: def increment(time, seconds):
    time.second += seconds
    if time.second >= 60:
        time.second -= 60
        time.minute += 1
    if time.minute >= 60:
        time.minute -= 60
        time.hour += 1
```

6. Define class and object, construct class called rectangle and initialize it with height=100,width=200. Write a program to display the center of a rectangle.

----- A class is a programmer-defined which acts as a blue print or a prototype.

Creating a new object is called instantiation, and the object is an instance of the class.

class Rectangle:

```
    box = Rectangle()
    box.width = 100.0
    box.height = 200.0
    box.corner = Point()
    box.corner.x = 0.0
    box.corner.y = 0.0
```

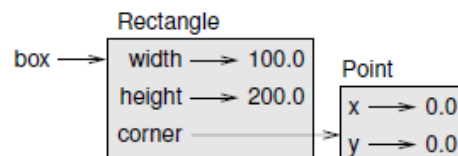


Figure 15.2: Object diagram.

7. Develop a program that uses class Student which prompts the user to enter marks in three subjects and calculates total marks, percentage and displays the score card details.

----- class Student:

```
    def __init__(self):
        self.name=input('Enter the student name: ')
        self.usn=input('Enter the student USN: ')
        self.marks =[]
    def getMarks(self):
        total = 0
        for i in range(3):
            self.marks.append(int(input('Enter the marks of subject '+str(i+1)+' ': )))
            total+=self.marks[i]
            self.marks.append(total)
    def Display(self):
        print('-----*****-----',end='\n\n')
        print('Student\'s score card: ')
```



```
        print('Student name:', self.name)
        print('Student USN:', self.usn)
        for i in range(3):
            print('subject '+str(i+1)+' marks:',self.marks[i])
            print ('Total marks: ',self.marks[3])
            print ('Percentage: ',self.marks[3]/3)
s=Student()
s.getMarks()
s.Display()
```

Output:

Enter the student name: Sahana
Enter the student USN: 054
Enter the marks of subject 1: 78
Enter the marks of subject 2: 67
Enter the marks of subject 3: 57